

While the *git* step is enough to handle most of the scenarios to download the source code, situations might arise where the download (which is essentially a Git clone) could fail if the source code repository is quite large, i.e., not just in terms of source code but other assets, like, binary files, system images, build packages, and extensive meta data, like branches and tags. The remedy for this is available as another step—*checkout*.

```
1. @echo
2.
3. src := $(wildcard src/*.* )
4. obj := $(src:.c=.o)
5.
6. pipeline: $(obj)
7.     $(cc) -o $@ $*
8.
9. .PHONY: clean
10.
11. clean:
12.     rm -f $(obj) pipimage
```

Figure 4: *makefile*

Compiling the source code

From the *dir*, *demo-sources*, we will call the *make* utility program to compile the code and generate the object artifact. The *make* utility uses a *makefile* that helps to build a tree of dependencies, define targets and variables, and perform actions. This *makefile* is also available in the root of the GitHub repo, mentioned in the section titled 'Build Flow'.

The source code is inside the *src* directory at the root of the GitHub repository. It is a tiny little program written in C language:

```
stage('Generate Binary') {
    dir('demo-sources') {
        sh "make pipimage"
    }
}
```

Archive

To enable the build artifact to be downloaded and used for further testing, we use the *archive* step to save the output from the previous compile stage. Once archived, the artifact can be downloaded from the *project* page in Jenkins for simple build projects. Jenkins supports integration with third-party artifact tools, like, for example, *JFrog* and *Maven*, which help to version and promote the artifacts.

```
stage('Archive') {
    dir('demo-sources') {
        archive excludes: '**.txt', includes: 'pipimage'
    }
}
```

The build artifact, thus generated, can be downloaded by clicking on the down arrow button that's available upon completing the build. Its details can also be checked on the *project* page.

Stage view

For every job run of the project that's triggered either manually or automatically, the Pipeline plugin presents a visual representation of all the completed and failed stages.



Figure 5: Build artifact



Figure 6: Artifact download



Figure 7: Pipeline stages

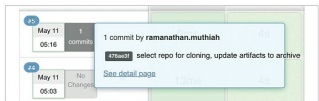


Figure 8: Commit info

If the build is successful, then the stages are shown in green, else, the failed stage is shown in red and the build stops there without executing any further stages in red.

If the Jenkins project is configured to trigger automatic builds on new commits in the configured GitHub repo, which can be enabled via WebHooks, then the commit information will be displayed along with the stages.

With this strategy to codify the entire build flow by breaking it up into its logical, constituent blocks and grouping them together, Jenkins Pipeline is in perfect alignment with the current trend to instrument the infrastructure using code. Of course, with the support of 1000+ plugins available in the ecosystem, there is a multitude of other possibilities to extend Jenkins capabilities, such as, to detect and set the build statuses, as well as send out notifications to Slack and HipChat. With BlueOcean, the revamped user interface (that offers the capability to write Jenkinsfile using declarative syntax), Jenkins is certainly evolving with the times.

For bigger projects, global Pipeline shared libraries reduce the maintenance burden by abstracting away the common tasks from the main Jenkinsfile. This approach brings down the entry barrier for engineering teams adapting or switching to Jenkins Pipeline. Refer to the Jenkins documentation at <https://jenkins.io/doc/book/pipeline/shared-libraries/>. 

By: Ramanathan Muthaiah

The author works for a cyber security firm in Bengaluru. He enjoys the outdoors when he's not churning out Jenkins Pipeline code.